

Introducción

AeroPredict Argentina: sistema inteligente para anticipar demanda aérea y detectar baja ocupación en rutas argentinas

1. Descripción general

AeroPredict Argentina es un sistema de inteligencia artificial orientado al análisis de la conectividad aérea en Argentina. Su propósito es utilizar datos históricos de vuelos, pasajeros y asientos para estimar la demanda futura de pasajeros y detectar situaciones relevantes para la toma de decisiones, como rutas con alta demanda o rutas con baja ocupación.

El sistema trabajará con información proveniente del dataset de Conectividad Aérea, que incluye datos relacionados con vuelos, aerolíneas, aeropuertos, pasajeros, asientos y rutas. A partir de estos datos, se buscará identificar patrones de comportamiento según variables como la fecha, el mes, el año, el origen, el destino, la aerolínea, la clasificación del vuelo y la cantidad de asientos disponibles.

2. Problema que busca resolver

Las aerolíneas, aeropuertos y áreas vinculadas al turismo necesitan anticiparse a los cambios en la demanda aérea. Si una ruta tendrá mucha demanda, puede ser necesario reforzar la oferta, monitorear la disponibilidad o planificar mejor los recursos. En cambio, si una ruta tiene muchos asientos disponibles pero pocos pasajeros, puede ser conveniente revisar promociones, frecuencias o estrategias turísticas.

Actualmente, tomar estas decisiones sin un análisis previo puede generar problemas como sobreoferta de asientos, baja ocupación, pérdida de oportunidades turísticas o falta de disponibilidad en temporadas de alta demanda.

3. Objetivo principal

El objetivo principal del sistema es predecir la cantidad esperada de pasajeros en rutas aéreas argentinas y detectar posibles casos de alta demanda o baja ocupación, utilizando datos históricos de vuelos, pasajeros y asientos.

3.1. Objetivos específicos

- Analizar la evolución histórica de pasajeros, vuelos y asientos.
- Calcular el factor de ocupación de las rutas mediante la fórmula: $\text{pasajeros} / \text{asientos}$.
- Identificar rutas con alta demanda de pasajeros.
- Detectar rutas con baja ocupación.
- Generar alertas que ayuden a interpretar los resultados del modelo.
- Brindar información útil para la planificación aérea, turística y comercial.

4. Resultado esperado

El sistema deberá permitir consultar información histórica y obtener predicciones o alertas. Por ejemplo, podría indicar que una ruta como Buenos Aires - Bariloche tiene alta demanda esperada en temporada turística, o que una ruta determinada presenta baja ocupación y podría necesitar acciones de promoción o revisión de frecuencia.

De esta manera, AeroPredict Argentina funcionará como una herramienta de apoyo a la toma de decisiones, combinando análisis de datos históricos con modelos predictivos de inteligencia artificial.

Funciones Principales

Funciones principales del sistema

Para que el proyecto no se vuelva DEMASIADO GRANDE ni difícil de organizar, va a ser mejor dividirlo en módulos. La idea es que cada módulo tenga una función clara y que después el backend y el frontend puedan trabajar con una estructura un poco más ordenada!

Módulo 1: Dashboard general

El primer módulo sería un dashboard. Esta sería la pantalla inicial del sistema, donde el usuario podría ver un resumen general de todo lo q es la información más importante

La idea es que, apenas se ingresa al sistema, se puedan ver datos básicos como la cantidad total de pasajeros, la cantidad total de vuelos, la cantidad total de asientos ofrecidos y el factor de ocupación promedio

Esto permitiría por ejemplo ver si en determinado período hubo más pasajeros que en otro, si aumentó la cantidad de vuelos o si la ocupación promedio fue alta o baja

También, en caso de q quieran, podría incluir un gráfico de evolución mensual de pasajeros, para observar de forma visual cómo fue cambiando la demanda a lo largo del tiempo. Esto es algo importante pq dependiendo el mes puede variar mucho, ya que puede ser temporada alta, vacaciones, feriados o períodos con menor movimiento

Las principales funciones de este módulo serían:

- **mostrar el total de pasajeros;**
- **mostrar el total de vuelos;**
- **mostrar el total de asientos;**
- **calcular y mostrar el factor de ocupación promedio;**
- **mostrar la evolución mensual de pasajeros.**

Módulo 2: Análisis de rutas

El segundo módulo estaría enfocado en analizar las rutas aéreas. Recordemos que el proyecto no solo busca ver datos generales, sino también entender qué rutas tienen más movimiento y cuáles podrían estar funcionando mejor o peor

En esta sección se podrían mostrar rankings de rutas, por ejemplo las 10 rutas con mayor cantidad de pasajeros. Esto permitiría identificar cuáles son los trayectos más importantes o más demandados dentro del mismo dataset

Además, está bueno pq se podría calcular el factor de ocupación por ruta. Clave pq no alcanza con saber cuántos pasajeros tuvo una ruta, también hay que compararlo con la cantidad de asientos ofrecidos. Una ruta puede tener muchos pasajeros, pero también muchísimos asientos disponibles, por lo que su ocupación podría no ser tan alta

También acá se permitiría detectar rutas con baja ocupación. Esto serviría para encontrar casos donde se ofrecen muchos asientos pero viajan pocos pasajeros. Desde un punto de vista turístico o comercial, estas rutas podrían necesitar promociones, cambios de frecuencia o una revisión de la oferta

A mi parecer, estaría bueno comparar vuelos de cabotaje con vuelos internacionales, para ver si se comportan de manera distinta. Por ejemplo, puede pasar que los vuelos de cabotaje tengan ciertos meses de mayor demanda por turismo interno, mientras que los internacionales respondan a otras temporadas. Esto solo es una idea, pero puede expandir un poco el alcance

Las principales funciones de este módulo serían:

- **mostrar las rutas con más pasajeros;**
- **mostrar las rutas con mejor factor de ocupación;**
- **detectar rutas con baja ocupación;**
- **comparar vuelos de cabotaje e internacionales;**
- **permitir analizar qué rutas tienen mayor o menor demanda.**

Módulo 3: Predicción de demanda

Acá entraría bien bien la IA

En esta parte, el sistema permitiría hacer una predicción de demanda a partir de datos históricos

La idea sería que el usuario pueda seleccionar o ingresar algunos datos, como una ruta, un mes, un año, la clasificación del vuelo o la cantidad de asientos, y que el sistema devuelva una estimación de cuántos pasajeros podrían esperarse

Por ejemplo, si se consulta una ruta como Buenos Aires - Bariloche para el mes de julio, el sistema podría predecir una demanda muy alta, ya que es una ruta asociada al turismo de invierno. En cambio, para otra ruta o período con menos movimiento, podría predecir una demanda media o baja.

Además de estimar pasajeros, el sistema también podría calcular una ocupación estimada. Esto permitiría saber si, según la cantidad de asientos disponibles, la ruta estaría cerca de llenarse o si tendría muchos lugares libres.

Para que el resultado sea más fácil de entender, el sistema no debería mostrar solamente un número. También podría clasificar la demanda en categorías simples como alta, media o baja. De esta manera, cualquier usuario puede interpretar el resultado sin necesidad de conocer detalles técnicos del modelo (usabilidad para todos)

Las principales funciones de este módulo serían:

- **permitir consultar una predicción para una ruta o período;**
- **estimar la cantidad esperada de pasajeros;**
- **estimar el factor de ocupación;**
- **clasificar la demanda como alta, media o baja;**

- **ayudar a anticipar períodos de mayor o menor movimiento.**

Módulo 4: Alertas inteligentes

El cuarto módulo estaría pensado para transformar los resultados del análisis y del modelo en alertas fáciles de interpretar

La idea es que el sistema no solo muestre datos, sino que también ayude a detectar situaciones importantes. Por ejemplo, si una ruta tiene una ocupación estimada muy alta, el sistema podría generar una alerta de **"alta demanda esperada"**. Esto podría servir para recomendar que se monitoree la disponibilidad o que se evalúe reforzar la oferta

En cambio, si una ruta tiene una ocupación baja, el sistema podría mostrar una alerta de **"baja ocupación"**. Este tipo de alerta serviría para detectar rutas donde quizás convenga hacer promociones, revisar la frecuencia de vuelos o analizar si la oferta de asientos es demasiado alta para la demanda real

También podrían aparecer alertas relacionadas con rutas turísticas que muestran crecimiento. Por ejemplo, si una ruta viene aumentando su cantidad de pasajeros en los últimos períodos, el sistema podría marcarla como una ruta con potencial turístico.

Las principales alertas podrían ser:

- **alta demanda esperada;**
- **baja ocupación;**
- **posible necesidad de promoción;**
- **ruta turística con crecimiento;**
- **ocupación estimada elevada.**

Módulo 5: Mapa o visualización de aeropuertos

OPCIONAL. SOLO SI HAY TIEMPO!

La idea sería agregar una visualización de aeropuertos en un mapa o en una sección gráfica

Podría usar el dataset de aeropuertos, que incluye información como el nombre del aeropuerto, código, localidad, provincia, país, latitud y longitud. Con esos datos se podría mostrar un mapa de Argentina con los aeropuertos y asociarles información del sistema, como cantidad de pasajeros, vuelos o nivel de demanda

Para mí este módulo sería como nuestro "valor diferencial", como que lo otro está bueno y es lo crucial para que funcione, pero este me parece que estaría bueno en caso de que se llegue

Por ejemplo, se podrían marcar aeropuertos con mayor movimiento o mostrar qué provincias tienen más actividad aérea. También se podría usar como complemento para entender mejor las rutas analizadas

Si no, la podemos dejar como “mejora futura”

Las principales funciones de este módulo serían:

- **mostrar aeropuertos de Argentina en un mapa;**
- **visualizar datos asociados a cada aeropuerto;**
- **identificar aeropuertos con mayor movimiento;**
- **complementar el análisis de rutas;**
- **mejorar la presentación visual del sistema.**

Datos (kiki)

Coordinación con el área de datos

Para poder terminar correctamente el diseño del sistema, es necesario coordinar con el área de datos qué información va a quedar disponible para backend, frontend y el modelo de inteligencia artificial.

A partir de la última actualización del equipo de datos, ya se definió que el archivo principal a utilizar será la **Base agregada de frecuencias aéreas** (`base_microdatos.csv`), perteneciente al dataset de **Conectividad Aérea**. Este archivo contiene información diaria sobre vuelos, pasajeros y asientos, organizada por origen, destino, aerolínea, clase de vuelo y clasificación del vuelo.

Esto permite avanzar con mayor seguridad en el diseño del sistema, ya que las variables principales que necesita AeroPredict Argentina están disponibles de forma directa o pueden construirse a partir del dataset.

Archivo principal confirmado

El archivo principal para el sistema será:

Base agregada de frecuencias aéreas (`base_microdatos.csv`)

Este archivo permite trabajar con:

- fecha;
- clasificación del vuelo;
- clase de vuelo;
- aerolínea;
- aeropuerto de origen;
- localidad de origen;
- provincia de origen;
- aeropuerto de destino;
- localidad de destino;
- provincia de destino;
- pasajeros;
- asientos;
- vuelos.

Estas variables son suficientes para construir las principales funciones del sistema: dashboard general, análisis de rutas, cálculo de ocupación, generación de alertas y entrenamiento del modelo predictivo.

Además, si el equipo llega con tiempo, se podrá usar el archivo auxiliar de **aeropuertos geolocalizados** para alimentar el módulo opcional del mapa de aeropuertos.

Variables disponibles y variables derivadas

Según lo definido por el área de datos, las variables que necesita diseño están cubiertas. Algunas vienen directamente en el archivo original y otras se pueden crear durante el proceso de limpieza y transformación.

Las variables disponibles directamente son:

- `indice_tiempo`;
- `clasificacion_vuelo`;
- `clase_vuelo`;
- `aerolinea`;
- `origen_aeropuerto`;
- `origen_localidad`;
- `origen_provincia`;
- `destino_aeropuerto`;
- `destino_localidad`;
- `destino_provincia`;
- `pasajeros`;
- `asientos`;
- `vuelos`.

A partir de esas columnas, el área de datos puede crear nuevas variables derivadas:

- `anio`, a partir de la fecha;
- `mes`, a partir de la fecha;
- `ruta`, combinando origen y destino;
- `factor_ocupacion`, calculado como pasajeros / asientos.

Por lo tanto, las variables que diseño necesita para las pantallas y endpoints principales sí pueden obtenerse del dataset.

Construcción de la columna ruta

Para el sistema es importante contar con una variable llamada `ruta`, ya que se va a usar para rankings, comparaciones, filtros, predicciones y alertas.

Esta columna se construirá combinando la localidad de origen y la localidad de destino. Por ejemplo:

`Buenos Aires - Bariloche`

Es importante aclarar que, según lo definido por datos, las rutas se van a mantener con dirección. Esto significa que:

`Buenos Aires - Bariloche`

y

Bariloche - Buenos Aires

serán consideradas rutas distintas.

Esta decisión es útil porque permite analizar la demanda real en cada sentido del trayecto.

Cálculo del factor de ocupación

El factor de ocupación será una de las métricas principales del sistema. Se calculará con la fórmula:

`factor_ocupacion = pasajeros / asientos`

Esta métrica permite saber qué proporción de los asientos ofrecidos fue ocupada por pasajeros.

Para evitar errores, el área de datos deberá controlar que la columna `asientos` exista, que no tenga valores problemáticos y que no se realicen divisiones por cero. Las filas con asientos iguales o menores a cero no deberían usarse para calcular ocupación.

En la base mensual por ruta, el factor de ocupación deberá calcularse sobre los totales mensuales:

`pasajeros totales del mes / asientos totales del mes`

Esto es mejor que promediar ocupaciones diarias, porque representa de forma más correcta la ocupación general del período.

Limpieza de datos necesaria

Antes de que backend pueda usar la información, el área de datos deberá entregar una base limpia y preparada.

La limpieza debería incluir:

- convertir correctamente la fecha;
- crear las columnas `año` y `mes`;
- normalizar nombres de localidades, provincias, aeropuertos y aerolíneas;
- revisar valores nulos o vacíos;
- descartar filas sin origen o destino;
- controlar pasajeros negativos;
- evitar divisiones por cero en el cálculo de ocupación;
- revisar casos sospechosos donde el factor de ocupación sea mayor a 1;
- excluir vuelos no comerciales o registros sin aerolínea válida, si corresponde;
- construir la columna `ruta`;
- documentar cuántas filas fueron descartadas o modificadas.

Esto es importante porque backend no debería recibir el archivo original lleno de inconsistencias, sino una base ya procesada y lista para consultar.

Base limpia esperada para backend

El área de datos debería entregar una base principal agregada por mes y ruta. Esta base sería el insumo más importante para los endpoints y también para el modelo predictivo.

La estructura esperada sería una fila por:

`anio + mes + ruta + clasificacion_vuelo`

Las columnas principales serían:

- `anio;`
- `mes;`
- `ruta;`
- `clasificacion_vuelo;`
- `origen_localidad;`
- `origen_provincia;`
- `destino_localidad;`
- `destino_provincia;`
- `pasajeros;`
- `asientos;`
- `vuelos;`
- `factor_ocupacion.`

Esta base permitiría resolver los endpoints principales del sistema, como resumen general, evolución de pasajeros, ranking de rutas, ocupación por ruta, rutas con baja ocupación y alertas.

Relación con backend y endpoints

Con la base mensual por ruta, backend podría responder los principales endpoints definidos desde diseño:

- `/api/resumen;`
- `/api/evolucion-pasajeros;`
- `/api/rutas/top;`
- `/api/rutas/ocupacion;`
- `/api/rutas/baja-ocupacion;`
- `/api/alertas.`

Además, esta misma base puede servir para entrenar el modelo de predicción de demanda, usando como variable objetivo la cantidad de pasajeros y como variables de entrada la ruta, el mes, el año, la clasificación del vuelo, los asientos y los vuelos.

Los endpoints de predicción, como `/api/prediccion/demanda` y `/api/prediccion/ocupacion`, dependerán de que el modelo esté entrenado y de que el equipo de IA confirme qué variables finales va a usar.

Umbrales iniciales para alertas

También se definieron algunos criterios iniciales que pueden usarse para generar alertas:

- baja ocupación: cuando el `factor_ocupacion` sea menor a 0.60;
- ocupación elevada: cuando el `factor_ocupacion` sea mayor o igual a 0.85;
- alta, media o baja demanda: según percentiles o comparación con el promedio histórico de cada ruta;
- ruta turística con crecimiento: cuando una ruta muestre un aumento sostenido de pasajeros en los últimos períodos.

Estos valores pueden ajustarse más adelante, pero sirven como punto de partida para que diseño, backend e IA trabajen con los mismos criterios.

Lo que queda pendiente validar

Aunque el archivo principal ya está definido y las variables necesarias están cubiertas, todavía queda una validación técnica pendiente por parte del área de datos.

Datos deberá descargar el CSV real y confirmar:

- que las columnas existan con los nombres esperados;
- que el rango de fechas sea correcto;
- que no haya cambios en la estructura del archivo;
- que los valores nulos o erróneos sean manejables;
- que la base limpia pueda generarse sin problemas;
- que el factor de ocupación quede dentro de valores razonables;
- que las rutas y localidades queden normalizadas;
- que backend pueda usar la base procesada sin hacer limpieza adicional.

Por lo tanto, desde diseño ya se puede avanzar con las pantallas y endpoints principales, pero la validación final del esquema real del CSV queda a cargo del área de datos.

Conclusión

Con la actualización del área de datos, queda confirmado que el proyecto puede avanzar usando la **Base agregada de frecuencias aéreas** como fuente principal. Esta base contiene las variables necesarias para analizar vuelos, pasajeros, asientos y rutas, y permite crear las variables derivadas que necesita el sistema, como `anio`, `mes`, `ruta` y `factor_ocupacion`.

De esta manera, AeroPredict Argentina puede sostener sus módulos principales: dashboard general, análisis de rutas, predicción de demanda y alertas inteligentes. El módulo de mapa

de aeropuertos queda como funcionalidad opcional, dependiendo de si el equipo llega a integrar el archivo auxiliar de aeropuertos geolocalizados.

Pantallas (niko)

Pantallas principales del sistema

El sistema AeroPredict Argentina se organizará en pantallas principales para que el usuario pueda consultar la información de manera clara y progresiva. La idea es que el sistema no muestre todos los datos juntos, sino que divida la información según el tipo de análisis que se quiere realizar.

Las pantallas propuestas son:

1. **Inicio / Dashboard general.**
2. **Rutas.**
3. **Predicción de demanda.**
4. **Alertas.**
5. **Mapa de aeropuertos, como módulo opcional si el equipo llega con tiempo.**

Esta división permite que el usuario primero tenga una mirada general del estado de la conectividad aérea, luego pueda analizar rutas específicas, después consultar predicciones y finalmente revisar alertas generadas por el sistema.

Pantalla 1: Inicio / Dashboard general

La primera pantalla del sistema será el **Inicio o Dashboard general**. Esta pantalla funcionará como el panel principal de AeroPredict Argentina, donde el usuario podrá ver un resumen rápido de la información más importante del sistema.

El objetivo de esta pantalla es que el usuario pueda entender, apenas ingresa, cuál fue el movimiento aéreo general del período analizado. Para eso, se mostrarán indicadores principales relacionados con pasajeros, vuelos, asientos y ocupación.

Esta pantalla es importante porque permite tener una primera lectura del comportamiento del transporte aéreo sin tener que entrar directamente en tablas o análisis más específicos.

Información que se mostraría

En esta pantalla se mostrarían los siguientes datos principales:

- pasajeros totales;
- vuelos totales;
- asientos totales;
- factor de ocupación promedio;
- evolución mensual de pasajeros;
- comparación entre cabotaje e internacional, si los datos lo permiten;
- alertas destacadas, como alta demanda o baja ocupación.

Los primeros cuatro datos podrían mostrarse como tarjetas o indicadores principales. Por ejemplo:

- **Pasajeros totales:** cantidad total de pasajeros transportados.
- **Vuelos totales:** cantidad total de vuelos registrados.
- **Asientos totales:** cantidad total de asientos ofrecidos.
- **Ocupación promedio:** relación entre pasajeros y asientos.

El factor de ocupación se calcularía como:

`factor_ocupacion = pasajeros / asientos`

Esto permite saber qué proporción de los asientos ofrecidos fue utilizada por pasajeros.

Gráficos propuestos

El gráfico principal de esta pantalla sería una **evolución mensual de pasajeros**. Este gráfico permitiría ver cómo cambia la demanda aérea a lo largo del tiempo.

Por ejemplo, se podrían observar:

- meses de mayor movimiento;
- meses de menor demanda;
- temporadas altas;
- caídas de pasajeros;
- diferencias entre vuelos de cabotaje e internacionales.

Este gráfico es importante porque la demanda aérea no suele ser igual durante todo el año. Puede variar por vacaciones, feriados, temporada turística, eventos o cambios económicos.

Filtros posibles

Para que el usuario pueda analizar mejor la información, el dashboard podría incluir filtros simples:

- año;
- mes;
- clasificación del vuelo, por ejemplo cabotaje o internacional;
- aerolínea, si se decide usar;
- provincia de origen;
- provincia de destino.

Estos filtros permitirían que el usuario no vea solamente los datos generales, sino también recortes específicos. Por ejemplo, podría consultar solo vuelos de cabotaje en 2024 o comparar la evolución de pasajeros internacionales.

Endpoints relacionados

Esta pantalla utilizaría principalmente los siguientes endpoints:

- GET /api/resumen;
- GET /api/evolucion-pasajeros;
- GET /api/alertas;
- GET /api/filtros.

El endpoint /api/resumen alimentaría las tarjetas principales.

El endpoint /api/evolucion-pasajeros permitiría construir el gráfico de evolución mensual.

El endpoint /api/alertas podría mostrar un resumen de las alertas más importantes.

El endpoint /api/filtros permitiría cargar las opciones disponibles para filtrar.

Justificación de la pantalla

Esta pantalla es necesaria porque funciona como la entrada principal al sistema. Antes de analizar una ruta o realizar una predicción, el usuario necesita una visión general del comportamiento de los datos.

Además, el dashboard ayuda a cumplir uno de los objetivos del proyecto: analizar la evolución histórica de pasajeros, vuelos y asientos. También permite detectar rápidamente si hubo aumento o caída de demanda en determinados períodos.

Pantalla 2: Rutas

La segunda pantalla estará dedicada al **análisis de rutas aéreas**. Esta sección permitirá identificar cuáles son las rutas con mayor movimiento, cuáles tienen mejor ocupación y cuáles presentan baja ocupación.

Esta pantalla es central para el sistema porque AeroPredict Argentina no busca analizar solamente datos generales, sino también entender el comportamiento de rutas específicas. Una ruta representa la conexión entre un origen y un destino, por ejemplo:

Buenos Aires - Bariloche

Según lo definido con el área de datos, la ruta se construirá combinando origen y destino. Además, se mantendrá la dirección del trayecto, por lo que:

Buenos Aires - Bariloche

y

Bariloche - Buenos Aires

serán consideradas rutas distintas.

Esto es importante porque la demanda puede no ser igual en ambos sentidos.

Información que se mostraría

En esta pantalla se mostrarían:

- top de rutas con más pasajeros;
- rutas con mayor factor de ocupación;
- rutas con menor factor de ocupación;
- cantidad de vuelos por ruta;
- cantidad de pasajeros por ruta;
- cantidad de asientos por ruta;
- factor de ocupación por ruta;
- comparación entre rutas de cabotaje e internacionales, si los datos lo permiten.

Ranking de rutas con más pasajeros

Uno de los elementos principales sería un ranking de las rutas con mayor cantidad de pasajeros.

Este ranking permitiría detectar cuáles son los trayectos con mayor demanda dentro del dataset. Por ejemplo, podría mostrar rutas turísticas o comerciales importantes.

Este dato es útil porque una ruta con muchos pasajeros puede requerir mayor seguimiento, especialmente en temporadas de alta demanda.

Análisis del factor de ocupación

Además del ranking por pasajeros, esta pantalla debería mostrar el factor de ocupación por ruta.

Esto es importante porque una ruta con muchos pasajeros no necesariamente tiene buena ocupación. Para saber si una ruta está funcionando bien, hay que comparar los pasajeros con la cantidad de asientos ofrecidos.

Por ejemplo:

- una ruta puede tener muchos pasajeros, pero también muchísimos asientos disponibles;
- otra ruta puede tener menos pasajeros, pero una ocupación muy alta porque casi todos los asientos se llenan.

Por eso, esta pantalla no debería mirar solo la demanda, sino también la relación entre demanda y oferta.

Rutas con baja ocupación

Otra parte importante de esta pantalla será la detección de rutas con baja ocupación.

Se podría considerar baja ocupación cuando:

```
factor_ocupacion < 0.60
```

Este valor se toma como umbral inicial y puede ajustarse más adelante.

Las rutas con baja ocupación son importantes porque pueden indicar que se ofrecen muchos asientos en relación con la cantidad de pasajeros. Desde el punto de vista turístico o comercial, estas rutas podrían necesitar promociones, revisión de frecuencia o un análisis más profundo.

Filtros posibles

La pantalla de rutas podría incluir filtros como:

- año;
- mes;
- clasificación del vuelo;
- origen;
- destino;
- provincia de origen;
- provincia de destino;
- aerolínea;
- nivel de ocupación.

Estos filtros permitirían que el usuario compare rutas en distintos períodos o tipos de vuelo.

Vista de detalle de una ruta

También sería útil que el usuario pueda seleccionar una ruta y ver un detalle más completo.

Por ejemplo, al seleccionar **Buenos Aires - Bariloche**, el sistema podría mostrar:

- evolución mensual de pasajeros de esa ruta;
- cantidad de vuelos;
- cantidad de asientos;
- factor de ocupación;
- meses con mayor demanda;
- meses con menor ocupación;
- alertas asociadas a esa ruta.

Esto permitiría pasar de un ranking general a un análisis más específico.

Endpoints relacionados

Esta pantalla utilizaría principalmente:

- GET /api/rutas/top;
- GET /api/rutas/ocupacion;
- GET /api/rutas/baja-ocupacion;
- GET /api/rutas/detalle;
- GET /api/filtros.

El endpoint /api/rutas/top serviría para mostrar las rutas con más pasajeros.

El endpoint /api/rutas/ocupacion permitiría ordenar rutas por ocupación.

El endpoint /api/rutas/baja-ocupacion mostraría los casos problemáticos.

El endpoint /api/rutas/detalle permitiría analizar una ruta puntual.

El endpoint /api/filtros cargaría las opciones disponibles para búsqueda y filtrado.

Justificación de la pantalla

Esta pantalla es necesaria porque el problema del sistema está directamente relacionado con la demanda y ocupación de rutas. No alcanza con saber cuántos pasajeros hubo en total; también hay que identificar dónde se concentra la demanda y qué rutas podrían presentar problemas de ocupación.

Además, esta pantalla permite cumplir varios objetivos específicos del proyecto:

- identificar rutas con alta demanda;
- detectar rutas con baja ocupación;
- comparar oferta y demanda;
- analizar diferencias entre rutas de cabotaje e internacionales;
- generar información útil para decisiones turísticas o comerciales.

Pantalla 3: Predicción de demanda

La tercera pantalla será la más relacionada con la inteligencia artificial. En esta sección, el usuario podrá consultar una predicción de demanda para una ruta o período específico.

El objetivo de esta pantalla es que el sistema no solo muestre datos históricos, sino que también pueda anticipar posibles comportamientos futuros.

Por ejemplo, el usuario podría consultar la ruta:

Buenos Aires - Bariloche

para el mes de julio, y el sistema podría devolver una estimación de pasajeros esperados, una ocupación estimada y una alerta de alta demanda.

Funcionamiento general

La pantalla tendría un formulario donde el usuario pueda ingresar o seleccionar los datos necesarios para realizar la predicción.

El formulario podría pedir:

- origen;
- destino;
- mes;
- año;
- clasificación del vuelo;
- aerolínea, si se decide usar;
- cantidad de asientos, si el modelo lo necesita;
- cantidad de vuelos, si el modelo lo necesita.

A partir de esos datos, el backend enviaría la consulta al modelo de inteligencia artificial. El modelo devolvería una estimación de pasajeros esperados.

Luego, si también se conoce la cantidad de asientos ofrecidos, el sistema podría calcular la ocupación estimada:

$$\text{ocupacion_estimada} = \text{pasajeros_predichos} / \text{asientos}$$

Resultado mostrado al usuario

El resultado no debería ser solamente un número. Para que sea fácil de interpretar, la pantalla debería mostrar:

- ruta consultada;
- período consultado;
- pasajeros esperados;
- ocupación estimada;
- nivel de demanda;
- nivel de ocupación;
- alerta generada;
- breve recomendación.

Por ejemplo:

Ruta	Mes	Pasajeros esperados	Ocupación estimada	Alerta

Buenos Aires - Bariloche	Julio	110.500	92%	Alta demanda esperada
--------------------------	-------	---------	-----	-----------------------

Clasificación de la demanda

Para que el resultado sea más comprensible, el sistema podría clasificar la demanda en niveles:

- demanda alta;
- demanda media;
- demanda baja.

Esta clasificación podría hacerse comparando la predicción con el comportamiento histórico de esa ruta. Por ejemplo, si la cantidad de pasajeros estimada supera el percentil 75 histórico, se podría marcar como demanda alta.

Clasificación de la ocupación

También se podría clasificar la ocupación estimada.

Un criterio inicial podría ser:

- **baja ocupación:** menor a 60%;
- **ocupación media:** entre 60% y 84%;
- **ocupación alta:** 85% o más.

Estos umbrales son simples, fáciles de explicar y sirven como primera versión del sistema.

Importancia de esta pantalla

Esta pantalla es importante porque representa el aporte principal de inteligencia artificial del proyecto. Las pantallas anteriores muestran información histórica, pero esta permite anticipar escenarios.

Esto puede ser útil para:

- monitorear rutas con alta demanda esperada;
- detectar posibles problemas de baja ocupación;
- planificar mejor la oferta de vuelos;
- decidir si conviene reforzar frecuencias;
- pensar promociones turísticas o comerciales.

Endpoints relacionados

Esta pantalla utilizaría principalmente:

- `POST /api/prediccion/demanda`;
- `POST /api/prediccion/ocupacion`;
- `GET /api/filtros`;
- `GET /api/modelo/metricas`, si se decide mostrar información técnica del modelo.

El endpoint `/api/prediccion/demanda` devolvería los pasajeros estimados.

El endpoint `/api/prediccion/ocupacion` devolvería la ocupación estimada.

El endpoint `/api/filtros` permitiría completar los campos del formulario con valores reales del dataset.

El endpoint `/api/modelo/metricas` podría mostrar información sobre el rendimiento del modelo, como error promedio o variables usadas.

Justificación de la pantalla

Esta pantalla es necesaria porque conecta directamente el sistema con el objetivo principal del proyecto: anticipar demanda aérea mediante inteligencia artificial.

Además, hace que AeroPredict Argentina no sea solamente un sistema descriptivo, sino también predictivo. Esto significa que no solo permite ver qué pasó, sino también estimar qué podría pasar.

Pantalla 4: Alertas

La cuarta pantalla estará dedicada a las **alertas inteligentes** generadas por el sistema.

La idea de esta pantalla es que el usuario no tenga que revisar todos los gráficos y tablas manualmente para encontrar situaciones importantes. En cambio, el sistema le mostrará directamente los casos que requieren atención.

Esta pantalla transforma los datos y predicciones en mensajes más fáciles de interpretar.

Tipos de alertas

Las alertas principales podrían ser:

- alta demanda esperada;
- baja ocupación;
- ocupación elevada;
- posible necesidad de promoción;
- ruta turística con crecimiento.

Cada alerta estaría asociada a una ruta, un período y una recomendación.

Ejemplos de alertas

Una alerta de alta demanda podría generarse cuando una ruta tiene una ocupación estimada muy alta o cuando los pasajeros esperados superan el comportamiento histórico normal.

Por ejemplo:

Buenos Aires - Bariloche | Julio | Alta demanda esperada

La recomendación podría ser:

Monitorear disponibilidad o evaluar refuerzo de oferta.

Una alerta de baja ocupación podría generarse cuando el factor de ocupación sea menor a 60%.

Por ejemplo:

Cordoba - Mendoza | Mayo | Baja ocupacion

La recomendación podría ser:

Evaluar promociones o revisar frecuencia.

Información que mostraría la tabla

La pantalla podría estar organizada como una tabla donde cada fila representa una alerta.

La tabla podría mostrar:

- ruta;
- mes o período;
- pasajeros reales o estimados;
- asientos;
- ocupación real o estimada;
- tipo de alerta;
- recomendación;
- clasificación del vuelo.

Ejemplo:

Ruta	Período	Pasajeros	Ocupación	Tipo de alerta	Recomendación
Buenos Aires - Bariloche	Julio	110.500	92%	Alta demanda	Monitorear disponibilidad
Cordoba - Mendoza	Mayo	25.000	41%	Baja ocupación	Evaluar promociones

Alertas históricas y predictivas

La pantalla podría mostrar dos tipos de alertas:

1. **Alertas históricas:** se generan a partir de datos reales ya registrados.
2. **Alertas predictivas:** se generan a partir de los resultados del modelo de inteligencia artificial.

En una primera versión, se pueden implementar alertas históricas usando el factor de ocupación real. Luego, cuando el modelo esté funcionando, se pueden agregar alertas predictivas.

Esto permite que el sistema tenga una versión inicial más simple, pero que pueda crecer más adelante.

Filtros posibles

La pantalla de alertas podría incluir filtros como:

- año;
- mes;
- tipo de alerta;
- ruta;
- clasificación del vuelo;
- nivel de ocupación;
- provincia de origen;
- provincia de destino.

Estos filtros ayudarían al usuario a encontrar rápidamente los casos que más le interesan.

Endpoints relacionados

Esta pantalla utilizaría principalmente:

- `GET /api/alertas`;
- `GET /api/rutas/baja-ocupacion`;
- `GET /api/filtros`.

El endpoint `/api/alertas` devolvería las alertas generales.

El endpoint `/api/rutas/baja-ocupacion` serviría para alimentar alertas relacionadas con baja ocupación.

El endpoint `/api/filtros` permitiría filtrar la tabla.

Justificación de la pantalla

Esta pantalla es necesaria porque resume los casos más importantes del sistema. En vez de que el usuario tenga que interpretar todos los datos manualmente, AeroPredict Argentina genera alertas que ayudan a tomar decisiones.

Además, esta pantalla refuerza el valor del proyecto como herramienta de apoyo para aerolíneas, aeropuertos o áreas vinculadas al turismo.

Pantalla 5: Mapa de aeropuertos

Esta pantalla sería opcional y se implementaría solo si el equipo llega con tiempo.

La idea sería mostrar un mapa con los aeropuertos de Argentina y asociarles información del sistema, como cantidad de pasajeros, vuelos, asientos o nivel de movimiento.

Para esta pantalla se podría usar el dataset complementario de aeropuertos geolocalizados, que incluye datos como:

- código del aeropuerto;
- nombre;
- localidad;
- provincia;
- país;
- latitud;
- longitud.

Información que se mostraría

El mapa podría mostrar:

- aeropuertos de Argentina;

- ubicación geográfica de cada aeropuerto;
- cantidad de pasajeros asociados;
- cantidad de vuelos;
- nivel de movimiento;
- provincia;
- localidad.

También podría permitir seleccionar un aeropuerto para ver más información.

Utilidad de esta pantalla

Esta pantalla no es indispensable para el funcionamiento principal del sistema, pero suma valor visual y ayuda a entender la conectividad aérea de forma geográfica.

Puede servir para identificar:

- aeropuertos con mayor movimiento;
- provincias con más actividad aérea;
- zonas con menor conectividad;
- relación entre aeropuertos y rutas principales.

Endpoint relacionado

Esta pantalla utilizaría principalmente:

- `GET /api/aeropuertos`.

Si el endpoint se combina con métricas de la base principal, también podría mostrar pasajeros o vuelos por aeropuerto.

Justificación de la pantalla

El mapa se plantea como mejora futura porque no es necesario para cumplir el objetivo principal del proyecto. Sin embargo, puede hacer que la presentación sea más visual y profesional.

Por eso, se lo considera un módulo opcional.

Relación entre pantallas y módulos del sistema

Pantalla	Módulo relacionado	Función principal
Inicio / Dashboard general	Dashboard general	Mostrar resumen del sistema y evolución de pasajeros.
Rutas	Análisis de rutas	Comparar rutas, pasajeros, asientos, vuelos y ocupación.
Predicción de demanda	Modelo de IA	Estimar pasajeros y ocupación esperada.
Alertas	Alertas inteligentes	Mostrar casos relevantes para la toma de decisiones.
Mapa de aeropuertos	Visualización opcional	Mostrar aeropuertos geolocalizados y movimiento asociado.

Conclusión

Las pantallas propuestas permiten organizar AeroPredict Argentina de manera clara y funcional. El sistema comenzaría con un dashboard general para mostrar la situación global, luego permitiría analizar rutas, consultar predicciones de demanda y revisar alertas inteligentes.

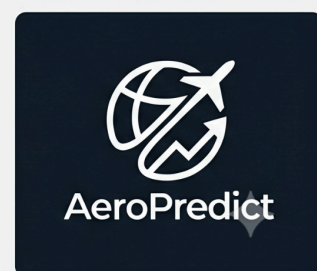
Esta estructura acompaña los objetivos principales del proyecto: analizar la evolución histórica de pasajeros, calcular ocupación, detectar rutas con baja ocupación, anticipar demanda y generar información útil para la toma de decisiones.

Además, el diseño queda conectado con los endpoints propuestos y con la base de datos limpia que debe entregar el área de datos. De esta manera, frontend, backend, datos e inteligencia artificial pueden trabajar sobre una estructura común y coherente.

diseños

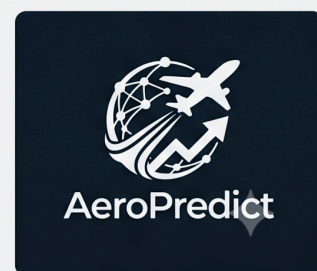
Identidad de Marca Minimalista – Logotipo Principal

Variaciones y Escalamiento Minimalista



Identidad de Marca – Logotipo Principal

Variaciones y Escalamiento



100

Predicción de demanda

Modelo activo (Entrenado: Jun 2025)

Completá los datos de una ruta y período para estimar pasajeros esperados, ocupación y alertas asociadas.

Consultar predicción

Ingresá los datos de la ruta y el período a analizar.

ORIGEN

Buenos Aires

DESTINO

Bariloche

MES

Julio

AÑO

2025

CLASIFICACIÓN

Cabotaje

ASIENTOS

120.000

VUELOS

800

Requerido para ocupación. Opcional, mejora precisión.

Predecir demanda

Limpiar consulta

Buenos Aires - Bariloche

Julio 2025

Cabotaje

Pasajeros esperados

110.500

Estimación del modelo IA

Ocupación estimada

92%

Nivel de demanda

Alta

Superior al promedio histórico

Alta demanda esperada

El modelo estima una demanda alta para esta ruta y período. La ocupación esperada (92%) supera el umbral de ocupación óptima. Se recomienda **monitorear disponibilidad y evaluar refuerzo de oferta de asientos** durante julio.

ocupación = $\frac{\text{pasajeros esperados}}{\text{asientos ofrecidos}}$

Contexto histórico de la ruta

Comparación de la predicción con el comportamiento habitual.

Simulación de escenarios

Escenario

Asientos

Ocupación

Alertas inteligentes

Actualizar alertas

Última actualización: hoy, 10:30

Modelo activo

Datos históricos y predictivos

Monitoreo de rutas con alta demanda, baja ocupación y oportunidades de acción

El sistema prioriza rutas y períodos que requieren atención para facilitar la toma de decisiones.

Filtros GET /api/filtros GET /api/alertas

Año Mes/Período Tipo de alert Tipo de alerta Todos/Cabotaje/Internacional Ruta Origen Destino Prioridad Aplicar filtros Limpiar Exportar reporte

Filtros: Cabotaje Baja ocupación 2024 Prioridad alta

Buscar ruta, origen o destino...

Alertas activas

36

En el período seleccionado

Alta demanda

12

Rutas con demanda elevada

Baja ocupación

18

Menor al 60%

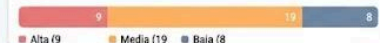
Rutas en crecimiento

6

Tendencia positiva reciente

Distribución por prioridad

La prioridad combina nivel de ocupación, demanda estimada y cantidad de pasajeros afectados.



Alertas detectadas

Todas Históricas Predictivas

Prioridad	Tipo de alerta	Ruta	Período	Pasajeros	Asientos	Ocupación	Clasificación	Recomendación	Estado	Acción
Alta	Alta demanda esperada	Córdoba - Mendoza	2024-05-08	234	130	60%	Baja Ocupación	Alta Ocupación	Nueva	Ver detalle
Alta	Alta demanda esperada	Buenos Aires - Bariloche	2024-05-08	150	150	85%	Histórica	Histórica	Pendiente	Ver detalle
Alta	Ocupación elevada	Buenos Aires - Iguazú	2024-05-09	198	150	85%	Histórica	Predictiva	En revisión	Ver detalle
Alta	Possible promoción	Buenos Aires - Salta	2024-05-09	89	130	60%	Ruta turística con crecimiento	Predictiva	En revisión	Ver detalle
Baja	Ruta en growth	Mendoza - Bariloche	2024-05-29	169	290	60%	Seguimiento recomendado	Predictiva	Revisada	Ver detalle

Criterios de detección

Las prioridades combinan niveles de ocupación afectados.

• Baja ocupación (< 60%)

• Ocupación elevada (> 85%)

• Alta demanda (pasajeros esperados históricos > histórica)

• Ruta in growth (sustained increase month)

• Possible promoción (sustained low occupancy OR demand < supply)

Los umbrales pueden ajustarse...

Recomendaciones panel

Las recomendaciones son orientativas y deben complementarse análisis comercial o turístico.

Monitorear disponibilidad

Evaluar promociones

Revisar frecuencia

Seguir crecimiento

Mapa de aeropuertos

Actualizar mapa

Visualización geográfica de conectividad aérea, movimiento y ocupación

Datos geolocalizados disponibles

Explorá aeropuertos de Argentina y visualizá su nivel de movimiento según pasajeros, vuelos y ocupación.

Año

Mes/Periodo

Clasificación de fligos

Provincia

Aeropuerto

Nivel de movimiento

Tipo de Alerta

2024

Meo

Todos / Cabotaje / Internacional

Provincia...

Alto movimiento

Alta ocupación

Alertas...

Aplicar filtros

Limpiar

Cabotaje

Buscar aeropuerto, localidad o provincia...

Centrar mapa

Cabotaje

Buenos Aires

Alta ocupación

2024

Clear filtros

Código: AEP

Nombre: Aeroparque Jorge Newbery

Localidad: Buenos Aires

Provincia: CABA

Pasajeros: 5.000.000

Vuelos: 42.000

Ocupación promedio: 84%

Estado: Alto movimiento

Ver detalle

Aeropuertos visualizados

58

Mayor movimiento

AEP (5.000.000 pasajeros)

Provincia destacada

Buenos Aires

Aeropuertos con alertas

9

Detalle de aeropuerto

AEP — Aeroparque Jorge Newbery

Locación

Aeroparque Jorge Newbery, CABA

Métricas principales

Pasajeros

Vuelos

Asientos

5.000.000

42.000

Ocupación promedio

Estado

Alto movimiento

Ocupación media-alta

Rutas principales compact

AEP — Aeroparque Jallieononta (CBR)

Buenos Aires — Heramjia Galena (RXI)

Buenos Aires — Povonia.CABA

Alertas asociadas

Alertas aeropuerto

Alertas modiamientos

Alertas basas asociadas

Ver rutas del aeropuerto

Ver alertas asociadas

Leyenda

El tamaño del marcador representa movimiento total. El color representa estado de ocupación o alerta.

Normal movimiento normal

Buan ocuparacy/growliento

Atención

Baja ocupancy/critios alert

Rutas del marcador com representa estado de ocupación o alerta.

Aeropuertos con mayor movimiento

Ver aeropuertos

Ver rutas principales

Mostrar alertas

Código	Aeropuerto	Provincia	Pasajeros	Vuelos	Ocupación	Estado	Acción
AEP	Aeroparque Jorge Newbery	Cabotaje	5.000.000	1.83.000	84%	alto movimiento	Ver detalle
EZE	Buenos Aires	CABA	3.000.000	42.000	42%	alta ocupación	Ver detalle
SLA	Buenos Aires	CABA	1.000.000	42.000	87%	atención	Ver detalle
UGN	Cuenos de Mercera	Buetos Aires	600.000	19.000	34%	baja ocupación	Ver deialle

Endpoints 10-06

Endpoints propuestos

Endpoints propuestos

Los endpoints del sistema AeroPredict Argentina se diseñan para conectar la base de datos procesada con las distintas pantallas del frontend y con el modelo de inteligencia artificial. La idea principal es que backend no trabaje directamente con el archivo CSV original, sino con una base limpia y preparada por el área de datos.

A partir de la coordinación con datos, se toma como fuente principal la Base agregada de frecuencias aéreas (`base_microdatos.csv`), perteneciente al dataset de Conectividad Aérea. Esta base permite trabajar con información de vuelos, pasajeros, asientos, aerolíneas, origen, destino, clasificación del vuelo y clase del vuelo.

Para que los endpoints sean más simples y eficientes, se propone que el área de datos entregue una base procesada y agregada por:

`anio + mes + ruta + clasificacion_vuelo`

Esto permite que backend pueda responder consultas del dashboard, rankings, ocupación, alertas y predicciones sin tener que limpiar el dataset original cada vez que se hace una consulta.

Datos centrales utilizados por los endpoints

Los endpoints se apoyan en las siguientes variables principales:

- `anio`;
- `mes`;
- `fecha` o `indice_tiempo`;
- `clasificacion_vuelo`;
- `clase_vuelo`;
- `aerolinea`;
- `origen_aeropuerto`;
- `origen_localidad`;
- `origen_provincia`;
- `destino_aeropuerto`;
- `destino_localidad`;
- `destino_provincia`;
- `pasajeros`;
- `asientos`;
- `vuelos`;

- `ruta`;
- `factor_ocupacion`.

Algunas de estas variables vienen directamente en el dataset y otras son creadas por el área de datos.

Por ejemplo:

- `anio` se obtiene a partir de la fecha;
- `mes` se obtiene a partir de la fecha;
- `ruta` se construye combinando origen y destino;
- `factor_ocupacion` se calcula a partir de pasajeros y asientos.

La fórmula principal utilizada es:

`factor_ocupacion = pasajeros / asientos`

Esta métrica es fundamental para el sistema, porque permite saber qué porcentaje de los asientos ofrecidos fue ocupado por pasajeros.

Criterios generales de diseño de la API

Los endpoints se dividen en tres grupos:

1. Endpoints principales, que alimentan las pantallas básicas del sistema.
2. Endpoints de inteligencia artificial, que dependen del modelo predictivo.
3. Endpoints opcionales o complementarios, que sirven para mejorar la experiencia del usuario, agregar filtros o sumar visualizaciones.

Se utilizan endpoints `GET` cuando el sistema solamente necesita consultar información ya existente, por ejemplo para mostrar un resumen, una evolución histórica o un ranking.

Se utilizan endpoints `POST` cuando el frontend necesita enviar datos al backend para que el modelo de inteligencia artificial procese una predicción. En estos casos conviene usar `POST` porque se manda un cuerpo de datos más completo, no solamente parámetros simples en la URL.

Tabla general de endpoints

Método	Endpoint	Para qué sirve	Pantalla relacionada	Estado

GET	/api/resumen	Devuelve métricas generales del sistema: pasajeros, vuelos, asientos y ocupación promedio.	Dashboard general	Principal
GET	/api/evolucion-pasajeros	Devuelve la evolución de pasajeros por mes o por año.	Dashboard general	Principal
GET	/api/rutas/top	Devuelve las rutas con mayor cantidad de pasajeros.	Rutas	Principal
GET	/api/rutas/ocupacion	Devuelve el factor de ocupación por ruta.	Rutas	Principal
GET	/api/rutas/baja-ocupacion	Devuelve rutas con ocupación menor a un umbral definido.	Rutas / Alertas	Principal
GET	/api/rutas/detalle	Devuelve información completa de una ruta específica.	Rutas	Recomendado
GET	/api/alertas	Devuelve alertas de alta demanda, baja ocupación o posible necesidad de promoción.	Alertas	Principal
POST	/api/prediccion/demanda	Recibe datos de una ruta o período y devuelve pasajeros estimados.	Predicción de demanda	Depende del modelo

POST	<code>/api/prediccion/ocupacion</code>	Recibe datos de una ruta o período y devuelve ocupación estimada.	Predicción de demanda	Depende del modelo
GET	<code>/api/filtros</code>	Devuelve valores disponibles para filtros: años, meses, rutas, aerolíneas, provincias y clasificación de vuelo.	Todas las pantallas	Recomendado
GET	<code>/api/aeropuertos</code>	Devuelve aeropuertos geolocalizados para mapa o filtros.	Mapa de aeropuertos	Opcional
GET	<code>/api/modelo/metricas</code>	Devuelve métricas básicas del modelo predictivo.	Predicción / documentación técnica	Opcional

Endpoints principales

1. GET `/api/resumen`

Este endpoint devuelve un resumen general del sistema. Se utiliza principalmente en la pantalla inicial o dashboard general.

Su objetivo es mostrar, de forma rápida, el estado general de la conectividad aérea analizada. El usuario puede ver cuántos pasajeros hubo, cuántos vuelos se registraron, cuántos asientos se ofrecieron y cuál fue el factor de ocupación promedio.

Justificación

Este endpoint es necesario porque el dashboard necesita mostrar información resumida sin obligar al usuario a revisar tablas o gráficos complejos. Funciona como una primera lectura del sistema.

Además, permite responder preguntas generales como:

- ¿Cuántos pasajeros se transportaron en el período seleccionado?
- ¿Cuántos vuelos se realizaron?
- ¿Cuántos asientos se ofrecieron?
- ¿Qué tan ocupados estuvieron los vuelos en promedio?

También es útil para comparar períodos. Por ejemplo, el usuario podría filtrar por año y ver si en 2024 hubo más pasajeros que en 2023.

Datos que utiliza

Este endpoint utiliza principalmente:

- `pasajeros`;
- `vuelos`;
- `asientos`;
- `factor_ocupacion`;
- `anio`;
- `mes`;
- `clasificacion_vuelo`.

Parámetros opcionales

Puede aceptar filtros para que el resumen no sea siempre global.

Ejemplo de filtros posibles:

GET /api/resumen?anio=2024&clasificacion_vuelo=Cabotaje

Parámetros:

Parámetro	Tipo	Obligatorio	Descripción
<code>anio</code>	número	No	Filtra los datos por año.
<code>mes</code>	número	No	Filtra los datos por mes.

clasificacion_vuelo	texto	No	Permite filtrar por cabotaje o internacional.
aerolinea	texto	No	Permite filtrar por aerolínea.
origen_provincia	texto	No	Permite filtrar por provincia de origen.
destino_provincia	texto	No	Permite filtrar por provincia de destino.

Ejemplo de respuesta

```
{
  "pasajeros_totales": 15000000,
  "vuelos_totales": 120000,
  "asientos_totales": 18000000,
  "factor_ocupacion_promedio": 0.83,
  "periodo": {
    "anio": 2024,
    "mes": null
  },
  "clasificacion_vuelo": "Cabotaje"
}
```

Explicación de la respuesta

- **pasajeros_totales**: suma total de pasajeros del período consultado.
- **vuelos_totales**: suma total de vuelos registrados.

- `asientos_totales`: suma total de asientos ofrecidos.
- `factor_ocupacion_promedio`: relación entre pasajeros y asientos.
- `periodo`: indica qué período se está consultando.
- `clasificacion_vuelo`: indica si el resumen corresponde a cabotaje, internacional o ambos.

Pantalla que alimenta

Este endpoint alimenta la pantalla:

Inicio / Dashboard general

En esa pantalla se usaría para mostrar tarjetas con indicadores principales.

2. GET `/api/evolucion-pasajeros`

Este endpoint devuelve la evolución de pasajeros a lo largo del tiempo. Sirve para construir gráficos de línea o gráficos de barras mensuales.

Justificación

Este endpoint es necesario porque el sistema no solo debe mostrar valores totales, sino también cómo cambiaron esos valores en el tiempo.

La evolución temporal es clave para detectar:

- temporadas altas;
- meses con baja demanda;
- crecimiento de pasajeros;
- caída de actividad;
- diferencias entre vuelos de cabotaje e internacionales;
- comportamiento turístico por época del año.

Por ejemplo, si los pasajeros aumentan en enero, julio o diciembre, el sistema puede relacionarlo con vacaciones, turismo o temporada alta.

Datos que utiliza

Este endpoint utiliza:

- `anio`;
- `mes`;
- `pasajeros`;
- `clasificacion_vuelo`;
- `ruta`, si se filtra una ruta específica.

Parámetros opcionales

GET /api/evolucion-pasajeros?anio=2024&clasificacion_vuelo=Cabotaje

Parámetro	Tipo	Obligatorio	Descripción
anio	número	No	Filtra la evolución por un año específico.
desde	texto	No	Fecha inicial en formato YYYY-MM.
hasta	texto	No	Fecha final en formato YYYY-MM.
clasificacion_vuelo	texto	No	Cabotaje o internacional.
ruta	texto	No	Permite ver la evolución de una ruta específica.
origen	texto	No	Filtra por localidad o aeropuerto de origen.
destino	texto	No	Filtra por localidad o aeropuerto de destino.

Ejemplo de uso

GET /api/evolucion-pasajeros?anio=2024&clasificacion_vuelo=Cabotaje

Ejemplo de respuesta

[

```
{
  "anio": 2024,
  "mes": 1,
  "mes_nombre": "Enero",
  "pasajeros": 1200000
},
{
  "anio": 2024,
  "mes": 2,
  "mes_nombre": "Febrero",
  "pasajeros": 1100000
},
{
  "anio": 2024,
  "mes": 3,
  "mes_nombre": "Marzo",
  "pasajeros": 980000
}
]
```

Explicación de la respuesta

Cada objeto representa un mes. El frontend puede usar estos datos para armar un gráfico de evolución mensual.

Pantalla que alimenta

Este endpoint alimenta la pantalla:

Inicio / Dashboard general

También puede usarse en la pantalla de rutas si se quiere ver la evolución de una ruta específica.

3. GET `/api/rutas/top`

Este endpoint devuelve las rutas con mayor cantidad de pasajeros.

Justificación

Este endpoint es necesario porque una de las funciones principales del sistema es identificar las rutas con más demanda.

No alcanza con saber la cantidad total de pasajeros del sistema. También es importante saber en qué rutas se concentra esa demanda.

Este endpoint permite responder preguntas como:

- ¿Cuáles son las rutas más importantes?
- ¿Qué destinos tienen mayor movimiento?
- ¿Qué rutas concentran más pasajeros?
- ¿Qué rutas podrían necesitar mayor monitoreo en temporada alta?

También es útil para generar gráficos de barras, rankings y comparaciones entre rutas.

Datos que utiliza

Este endpoint utiliza:

- `ruta`;
- `origen_localidad`;
- `destino_localidad`;
- `pasajeros`;
- `vuelos`;
- `asientos`;
- `factor_ocupacion`;
- `clasificacion_vuelo`;
- `anio`;
- `mes`.

Parámetros opcionales

GET `/api/rutas/top?limite=10&anio=2024&clasificacion_vuelo=Cabotaje`

Parámetro	Tipo	Obligatorio	Descripción
limite	número	No	Cantidad de rutas a devolver. Por defecto podría ser 10.
anio	número	No	Filtra por año.
mes	número	No	Filtra por mes.
clasificacion_vuelo	texto	No	Cabotaje o internacional.
orden	texto	No	Permite ordenar por pasajeros, vuelos, asientos u ocupación.

Ejemplo de respuesta

```
[
  {
    "ruta": "Buenos Aires - Bariloche",
    "origen": "Buenos Aires",
    "destino": "Bariloche",
    "pasajeros": 850000,
    "vuelos": 5200,
    "asientos": 950000,
    "factor_ocupacion": 0.89
  },
  {
```

```
"ruta": "Buenos Aires - Mendoza",  
"origen": "Buenos Aires",  
"destino": "Mendoza",  
"pasajeros": 790000,  
"vuelos": 4800,  
"asientos": 900000,  
"factor_ocupacion": 0.87  
}  
]
```

Explicación de la respuesta

Cada objeto representa una ruta. Se muestran pasajeros, vuelos, asientos y ocupación para que el usuario pueda comparar no solo la demanda, sino también la oferta y el rendimiento de cada ruta.

Pantalla que alimenta

Este endpoint alimenta la pantalla:

Rutas

Principalmente se usaría para mostrar el ranking de rutas con más pasajeros.

4. GET `/api/rutas/ocupacion`

Este endpoint devuelve el factor de ocupación por ruta.

Justificación

Este endpoint es clave porque una ruta con muchos pasajeros no necesariamente es una ruta eficiente. Para interpretar correctamente el rendimiento de una ruta, hay que comparar pasajeros con asientos ofrecidos.

Por ejemplo, una ruta puede tener muchos pasajeros, pero si también tiene demasiados asientos disponibles, su ocupación puede ser baja. En cambio, una ruta con menos pasajeros pero con pocos asientos ofrecidos puede tener una ocupación alta.

Este endpoint permite analizar:

- rutas con buena ocupación;
- rutas con baja ocupación;
- rutas con exceso de oferta;
- rutas donde la demanda aprovecha bien los asientos disponibles.

Datos que utiliza

Este endpoint utiliza:

- ruta;
- pasajeros;
- asientos;
- factor_ocupacion;
- vuelos;
- anio;
- mes;
- clasificacion_vuelo.

Parámetros opcionales

GET /api/rutas/ocupacion?anio=2024&orden=desc

Parámetro	Tipo	Obligatorio	Descripción
anio	número	No	Filtra por año.
mes	número	No	Filtra por mes.
clasificacion_vuelo	texto	No	Cabotaje o internacional.
orden	texto	No	Puede ser asc o desc.

limite	número	No	Cantidad máxima de rutas a devolver.
--------	--------	----	--------------------------------------

Ejemplo de respuesta

```
[
  {
    "ruta": "Buenos Aires - Bariloche",
    "pasajeros": 850000,
    "asientos": 950000,
    "vuelos": 5200,
    "factor_ocupacion": 0.89
  },
  {
    "ruta": "Cordoba - Mendoza",
    "pasajeros": 25000,
    "asientos": 60000,
    "vuelos": 420,
    "factor_ocupacion": 0.41
  }
]
```

Explicación de la respuesta

El campo más importante es **factor_ocupacion**, que indica qué proporción de los asientos ofrecidos fue ocupada.

Un valor de **0.89** equivale aproximadamente a 89% de ocupación.

Un valor de **0.41** equivale aproximadamente a 41% de ocupación.

Pantalla que alimenta

Este endpoint alimenta la pantalla:

Rutas

También puede alimentar la pantalla de alertas, porque las rutas con ocupación muy baja o muy alta pueden generar alertas automáticas.

5. GET `/api/rutas/baja-ocupacion`

Este endpoint devuelve las rutas cuyo factor de ocupación está por debajo de un umbral definido.

Justificación

Este endpoint es uno de los más importantes del sistema, porque se relaciona directamente con el problema que AeroPredict Argentina busca resolver: detectar rutas con baja ocupación.

Una baja ocupación puede indicar que se están ofreciendo muchos asientos para una demanda que no alcanza a cubrirlos. Esto puede ser útil para que aerolíneas, aeropuertos o áreas de turismo revisen la oferta, planifiquen promociones o analicen la frecuencia de vuelos.

El objetivo no es eliminar rutas, sino detectar casos que merecen análisis.

Criterio inicial

Se propone tomar como baja ocupación:

`factor_ocupacion < 0.60`

Es decir, rutas donde se ocupa menos del 60% de los asientos ofrecidos.

Este umbral puede modificarse más adelante según lo que decida el equipo o según el análisis de datos reales.

Datos que utiliza

Este endpoint utiliza:

- `ruta`;
- `pasajeros`;
- `asientos`;
- `vuelos`;

- `factor_ocupacion;`
- `anio;`
- `mes;`
- `clasificacion_vuelo.`

Parámetros opcionales

GET /api/rutas/baja-ocupacion?umbral=0.60&anio=2024

Parámetro	Tipo	Obligatorio	Descripción
<code>umbral</code>	decimal	No	Valor máximo para considerar baja ocupación. Por defecto puede ser <code>0.60</code> .
<code>anio</code>	número	No	Filtra por año.
<code>mes</code>	número	No	Filtra por mes.
<code>clasificacion_vuelo</code>	texto	No	Cabotaje o internacional.
<code>limite</code>	número	No	Cantidad máxima de rutas a devolver.

Ejemplo de respuesta

```
[
  {
    "ruta": "Cordoba - Mendoza",
    "pasajeros": 25000,
```

```
"asientos": 60000,  
"vuelos": 420,  
"factor_ocupacion": 0.41,  
"alerta": "Baja ocupacion",  
"recomendacion": "Evaluar promociones o revisar frecuencia"  
}  
]
```

Explicación de la respuesta

El endpoint no solo devuelve los datos de la ruta, sino también una alerta y una recomendación simple. Esto ayuda a que el resultado sea más entendible para usuarios no técnicos.

Pantallas que alimenta

Este endpoint alimenta dos pantallas:

- Rutas, para mostrar rutas con peor ocupación.
- Alertas, para mostrar casos que requieren atención.

6. GET `/api/rutas/detalle`

Este endpoint devuelve información completa de una ruta específica.

Justificación

Este endpoint no estaba en la primera versión, pero es recomendable agregarlo porque permite profundizar el análisis de una ruta puntual.

Los endpoints anteriores muestran rankings o listados generales. Sin embargo, si el usuario hace clic en una ruta, el sistema debería poder mostrar más detalle.

Por ejemplo, si el usuario selecciona la ruta **Buenos Aires - Bariloche**, este endpoint puede devolver:

- evolución mensual de pasajeros;
- vuelos totales;
- asientos ofrecidos;

- factor de ocupación;
- meses con mayor demanda;
- meses con menor ocupación;
- clasificación del vuelo;
- alertas asociadas.

Esto mejora la experiencia del usuario porque permite pasar de una vista general a una vista específica.

Datos que utiliza

Este endpoint utiliza:

- ruta;
- anio;
- mes;
- pasajeros;
- asientos;
- vuelos;
- factor_ocupacion;
- origen_localidad;
- destino_localidad;
- clasificacion_vuelo.

Parámetros

GET /api/rutas/detalle?ruta=Buenos%20Aires%20-%20Bariloche&anio=2024

Parámetro	Tipo	Obligatorio	Descripción
ruta	texto	Sí	Ruta a consultar.
anio	número	No	Filtra por año.
clasificacion_vuelo	texto	No	Cabotaje o internacional.

Ejemplo de respuesta

```
{
  "ruta": "Buenos Aires - Bariloche",
  "origen": "Buenos Aires",
  "destino": "Bariloche",
  "resumen": {
    "pasajeros_totales": 850000,
    "vuelos_totales": 5200,
    "asientos_totales": 950000,
    "factor_ocupacion_promedio": 0.89
  },
  "evolucion_mensual": [
    {
      "anio": 2024,
      "mes": 1,
      "mes_nombre": "Enero",
      "pasajeros": 95000,
      "asientos": 105000,
      "factor_ocupacion": 0.90
    },
    {
      "anio": 2024,
      "mes": 7,
      "mes_nombre": "Julio",
      "pasajeros": 120000,
      "asientos": 130000,
```

```
    "factor_ocupacion": 0.92
  }
],
"alertas": [
  {
    "tipo_alerta": "Alta demanda",
    "periodo": "Julio",
    "recomendacion": "Monitorear disponibilidad en temporada alta"
  }
]
}
```

Pantalla que alimenta

Este endpoint alimenta la pantalla:

Rutas

Especialmente si se agrega una vista de detalle o modal de una ruta seleccionada.

7. GET `/api/alertas`

Este endpoint devuelve alertas generadas a partir de los datos históricos o de los resultados del modelo predictivo.

Justificación

Este endpoint transforma los datos en información útil para la toma de decisiones.

El sistema no debería limitarse a mostrar números. Su valor está en interpretar esos números y generar alertas simples, como:

- alta demanda esperada;
- baja ocupación;
- posible necesidad de promoción;
- ocupación elevada;

- ruta turística con crecimiento.

Esto hace que AeroPredict Argentina funcione como una herramienta inteligente y no solo como un tablero de estadísticas.

Tipos de alertas posibles

Tipo de alerta	Criterio posible	Significado
Alta demanda	Pasajeros por encima del promedio histórico o percentil 75	La ruta o período tiene mucha demanda.
Baja ocupación	Factor de ocupación menor a 0.60	Hay muchos asientos ofrecidos en relación con los pasajeros.
Ocupación elevada	Factor de ocupación mayor o igual a 0.85	La ruta está cerca de llenarse.
Posible promoción	Baja ocupación sostenida	Podría necesitar acciones comerciales o turísticas.
Ruta con crecimiento	Aumento sostenido de pasajeros	Puede ser una ruta turística o comercial en expansión.

Datos que utiliza

Este endpoint utiliza:

- ruta;
- año;
- mes;
- pasajeros;
- asientos;
- vuelos;
- factor_ocupacion;
- predicción de pasajeros, si el modelo está disponible;
- predicción de ocupación, si el modelo está disponible.

Parámetros opcionales

GET /api/alertas?anio=2024&tipo_alerta=Baja%20ocupacion

Parámetro	Tipo	Obligatorio	Descripción
anio	número	No	Filtra por año.
mes	número	No	Filtra por mes.
tipo_alerta	texto	No	Filtra por tipo de alerta.
clasificacion_vuelo	texto	No	Cabotaje o internacional.
limite	número	No	Cantidad máxima de alertas a devolver.

Ejemplo de respuesta

```
[
  {
    "ruta": "Buenos Aires - Bariloche",
    "periodo": "Julio 2024",
    "pasajeros": 120000,
    "asientos": 130000,
    "ocupacion": 0.92,
    "tipo_alerta": "Alta demanda",
    "recomendacion": "Monitorear disponibilidad o evaluar refuerzo de oferta"
```

```
},  
  
{  
  "ruta": "Cordoba - Mendoza",  
  "periodo": "Mayo 2024",  
  "pasajeros": 25000,  
  "asientos": 60000,  
  "ocupacion": 0.41,  
  "tipo_alerta": "Baja ocupacion",  
  "recomendacion": "Evaluar promociones o revisar frecuencia"  
}  
]
```

Explicación de la respuesta

Cada alerta incluye:

- la ruta afectada;
- el período;
- los valores principales;
- el tipo de alerta;
- una recomendación simple.

Pantalla que alimenta

Este endpoint alimenta la pantalla:

Alertas

También puede usarse para mostrar una vista resumida de alertas dentro del dashboard.

Endpoints de inteligencia artificial

Estos endpoints representan la parte predictiva del sistema. Todavía dependen de que el modelo esté entrenado y de que el equipo confirme las variables finales que usará.

La idea es que el modelo aprenda a partir de datos históricos y pueda estimar:

- cantidad esperada de pasajeros;
 - ocupación esperada;
 - nivel de demanda;
 - alertas asociadas.
-

8. POST `/api/prediccion/demanda`

Este endpoint recibe datos de una ruta o período y devuelve una predicción de pasajeros esperados.

Justificación

Este endpoint representa el objetivo principal de inteligencia artificial del proyecto: anticipar la demanda aérea.

Mientras los endpoints anteriores muestran información histórica, este endpoint busca estimar qué podría pasar en un período futuro o en una consulta específica.

Es importante porque permite responder preguntas como:

- ¿Cuántos pasajeros podrían esperarse en una ruta determinada?
- ¿Qué rutas podrían tener alta demanda?
- ¿Qué meses conviene monitorear?
- ¿Dónde podría necesitarse mayor oferta?

Por qué se usa POST

Se usa `POST` porque el frontend necesita enviar varios datos al backend para que el modelo realice la predicción. Estos datos no son solamente filtros, sino variables de entrada para un modelo de inteligencia artificial.

Además, usar `POST` permite enviar un cuerpo JSON más claro y flexible.

Variables de entrada posibles

El modelo podría recibir:

- `origen`;
- `destino`;
- `ruta`;
- `mes`;
- `anio`;
- `clasificacion_vuelo`;

- **aerolinea**, si se decide usar;
- **asientos**;
- **vuelos**.

No todas las variables tienen que ser obligatorias en la primera versión. Eso dependerá del modelo final.

Ejemplo de pedido

```
{  
  "origen": "Buenos Aires",  
  "destino": "Bariloche",  
  "mes": 7,  
  "anio": 2025,  
  "clasificacion_vuelo": "Cabotaje",  
  "asientos": 120000,  
  "vuelos": 800  
}
```

Ejemplo de respuesta

```
{  
  "ruta": "Buenos Aires - Bariloche",  
  "pasajeros_predichos": 110500,  
  "nivel_demanda": "Alta",  
  "alerta": "Alta demanda esperada",  
  "recomendacion": "Monitorear disponibilidad o evaluar refuerzo de oferta"  
}
```

Criterio para nivel de demanda

El campo `nivel_demanda` puede calcularse comparando los pasajeros predichos con el comportamiento histórico de esa ruta.

Una forma simple sería:

- Alta demanda: si la predicción supera el percentil 75 histórico.
- Demanda media: si la predicción se ubica entre valores normales.
- Baja demanda: si la predicción queda por debajo del promedio o de un umbral definido.

Pantalla que alimenta

Este endpoint alimenta la pantalla:

Predicción de demanda

El usuario podría completar un formulario y recibir una estimación de pasajeros.

9. POST `/api/prediccion/ocupacion`

Este endpoint recibe datos de una ruta o período y devuelve una ocupación estimada.

Justificación

Este endpoint complementa la predicción de demanda.

Predecir pasajeros es útil, pero no alcanza para saber si una ruta estará bien aprovechada. Para eso se necesita comparar la cantidad de pasajeros esperados con la cantidad de asientos ofrecidos.

Por eso, este endpoint calcula la ocupación estimada:

```
ocupacion_predicha = pasajeros_predichos / asientos
```

Esto permite detectar si una ruta podría estar cerca de llenarse o si podría tener muchos asientos libres.

Por qué se usa POST

Se usa `POST` porque el frontend envía variables para que el backend calcule una predicción. Además, algunas variables pueden ser numéricas y otras categóricas, por lo que conviene enviarlas en un cuerpo JSON.

Variables de entrada posibles

- `origen;`

- destino;
- ruta;
- mes;
- anio;
- clasificacion_vuelo;
- asientos;
- vuelos;
- aerolinea, si se decide usar.

Ejemplo de pedido

```
{  
  "origen": "Buenos Aires",  
  "destino": "Bariloche",  
  "mes": 7,  
  "anio": 2025,  
  "clasificacion_vuelo": "Cabotaje",  
  "asientos": 120000,  
  "vuelos": 800  
}
```

Ejemplo de respuesta

```
{  
  "ruta": "Buenos Aires - Bariloche",  
  "pasajeros_predichos": 110500,  
  "asientos": 120000,  
  "ocupacion_predicha": 0.92,  
  "nivel_ocupacion": "Alta",  
  "alerta": "Alta ocupacion esperada",  
  "recomendacion": "Monitorear disponibilidad o evaluar refuerzo de oferta"
```

}

Criterio para nivel de ocupación

Se pueden usar los siguientes umbrales iniciales:

Nivel	Criterio
Baja	menor a 0.60
Media	entre 0.60 y 0.84
Alta	mayor o igual a 0.85

Estos umbrales son simples y fáciles de explicar. Más adelante podrían ajustarse con análisis estadístico.

Pantalla que alimenta

Este endpoint alimenta la pantalla:

Predicción de demanda

También puede alimentar la pantalla de alertas si se desea mostrar alertas predictivas.

Endpoints recomendados para mejorar el sistema

10. GET `/api/filtros`

Este endpoint devuelve los valores disponibles para construir filtros en el frontend.

Justificación

Aunque no formaba parte de la primera lista, es muy recomendable incluirlo porque facilita mucho el trabajo del frontend.

El frontend necesita saber qué años, meses, rutas, aerolíneas, provincias o clasificaciones de vuelo existen en la base. Sin este endpoint, habría que escribir esos valores manualmente, lo cual puede generar errores.

Este endpoint ayuda a que los filtros sean dinámicos y estén conectados con los datos reales.

Datos que utiliza

Utiliza valores únicos de columnas como:

- `anio;`
- `mes;`
- `ruta;`
- `clasificacion_vuelo;`
- `aerolinea;`
- `origen_provincia;`
- `destino_provincia;`
- `origen_localidad;`
- `destino_localidad.`

Ejemplo de uso

GET /api/filtros

Ejemplo de respuesta

```
{
  "anios": [2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024],
  "meses": [
    {
      "numero": 1,
      "nombre": "Enero"
    },
  ],
}
```

```
{
  "numero": 2,
  "nombre": "Febrero"
},
"clasificacion_vuelo": ["Cabotaje", "Internacional"],
"rutas": [
  "Buenos Aires - Bariloche",
  "Buenos Aires - Mendoza",
  "Cordoba - Mendoza"
],
"aerolineas": [
  "Aerolineas Argentinas",
  "Flybondi",
  "JetSmart"
],
"provincias": [
  "Buenos Aires",
  "Cordoba",
  "Mendoza",
  "Rio Negro"
]
}
```

Pantallas que alimenta

Este endpoint puede alimentar todas las pantallas:

- dashboard;
 - rutas;
 - predicción;
 - alertas;
 - mapa.
-

11. GET `/api/modelo/metricas`

Este endpoint devuelve información básica sobre el rendimiento del modelo de inteligencia artificial.

Justificación

Este endpoint es opcional, pero suma valor si se quiere justificar mejor la parte de IA.

No alcanza con decir que el sistema predice. También es importante mostrar, aunque sea de forma simple, cómo rindió el modelo.

Este endpoint podría devolver métricas como:

- error promedio;
- porcentaje de error;
- cantidad de registros usados para entrenamiento;
- fecha de entrenamiento;
- variables usadas.

Esto ayuda a explicar que el modelo fue evaluado y que sus resultados no salen de forma arbitraria.

Datos que utiliza

Este endpoint usa información generada durante el entrenamiento del modelo.

Puede incluir:

- MAE;
- RMSE;
- MAPE;
- cantidad de registros de entrenamiento;
- variables usadas;
- fecha de entrenamiento.

Ejemplo de uso

GET `/api/modelo/metricas`

Ejemplo de respuesta

```
{  
  "modelo": "Random Forest Regressor",  
  "variable_objetivo": "pasajeros",  
  "registros_entrenamiento": 85000,  
  "mae": 12500,  
  "rmse": 18000,  
  "mape": 0.08,  
  "fecha_entrenamiento": "2025-06-01",  
  "variables_usadas": [  
    "anio",  
    "mes",  
    "ruta",  
    "clasificacion_vuelo",  
    "asientos",  
    "vuelos"  
  ]  
}
```

Explicación de la respuesta

- **modelo**: indica qué algoritmo se utilizó.
- **variable_objetivo**: indica qué se está prediciendo.
- **mae**: error absoluto medio.
- **rmse**: error cuadrático medio.
- **mape**: error porcentual medio.
- **variables_usadas**: muestra qué columnas se usaron para entrenar.

Pantalla que alimenta

Este endpoint no necesariamente necesita una pantalla propia. Puede usarse en la documentación del sistema o en una sección técnica de la pantalla de predicción.

Endpoint opcional

12. GET `/api/aeropuertos`

Este endpoint devuelve información de aeropuertos para usar en un mapa o en filtros visuales.

Justificación

Este endpoint no es obligatorio para la primera versión del sistema, pero puede funcionar como mejora visual.

Si se utiliza el dataset de aeropuertos geolocalizados, el frontend podría mostrar un mapa de Argentina con los aeropuertos y asociar información como pasajeros, vuelos, demanda u ocupación.

Esto le daría al sistema un valor visual más fuerte, especialmente en una presentación.

Datos que utiliza

Este endpoint utilizaría el dataset complementario de aeropuertos, que puede incluir:

- código del aeropuerto;
- nombre;
- localidad;
- provincia;
- país;
- latitud;
- longitud.

También podría combinarse con la base principal para agregar información de movimiento.

Parámetros opcionales

GET `/api/aeropuertos?provincia=Buenos%20Aires`

Parámetro	Tipo	Obligatorio	Descripción
provincia	texto	No	Filtra aeropuertos por provincia.
pais	texto	No	Filtra por país.
con_mtricas	booleano	No	Si es verdadero, agrega pasajeros, vuelos o demanda asociada.

Ejemplo de respuesta simple

```
[
  {
    "codigo": "AEP",
    "nombre": "Aeroparque Jorge Newbery",
    "localidad": "Buenos Aires",
    "provincia": "Ciudad Autonoma de Buenos Aires",
    "pais": "Argentina",
    "latitud": -34.5592,
    "longitud": -58.4156
  }
]
```

Ejemplo de respuesta con métricas

```
[
  {
    "codigo": "AEP",
```

```
"nombre": "Aeroparque Jorge Newbery",  
"localidad": "Buenos Aires",  
"provincia": "Ciudad Autonoma de Buenos Aires",  
"pais": "Argentina",  
"latitud": -34.5592,  
"longitud": -58.4156,  
"pasajeros_totales": 5000000,  
"vuelos_totales": 42000,  
"nivel_movimiento": "Alto"  
}  
]
```

Pantalla que alimenta

Este endpoint alimenta la pantalla opcional:

Mapa o visualización de aeropuertos

Si no se llega a implementar, puede quedar como mejora futura.

Validaciones generales de los endpoints

Para que los endpoints funcionen correctamente, backend debería aplicar algunas validaciones básicas.

Validaciones sobre fechas

Los parámetros **anio**, **mes**, **desde** y **hasta** deben tener valores válidos.

Ejemplos:

- **anio** debe ser un número válido.
- **mes** debe estar entre 1 y 12.
- **desde** y **hasta** deben respetar el formato **YYYY-MM**.

Esto evita errores en consultas temporales.

Validaciones sobre ocupación

El factor de ocupación solo debe calcularse cuando:

`asientos > 0`

Si los asientos son cero o están vacíos, no se debería calcular la ocupación porque generaría una división inválida.

Validaciones sobre rutas

La ruta debe construirse siempre con origen y destino.

Ejemplo:

`origen_localidad + " - " + destino_localidad`

También se debe respetar la dirección de la ruta. Es decir:

`Buenos Aires - Bariloche`

no es lo mismo que:

`Bariloche - Buenos Aires`

Esto permite analizar la demanda en cada sentido.

Validaciones sobre predicción

En los endpoints de predicción, backend debe revisar que se reciban los datos mínimos necesarios.

Por ejemplo:

- origen o ruta;
- destino, si no se manda ruta completa;
- mes;
- año;
- clasificación del vuelo;
- asientos, si se va a calcular ocupación;
- vuelos, si el modelo lo requiere.

Si falta un dato obligatorio, el endpoint debería devolver un error claro.

Respuestas de error sugeridas

Para que el sistema sea más ordenado, los endpoints deberían devolver errores con una estructura clara.

Error 400: datos inválidos

Se usaría cuando el usuario manda parámetros incorrectos o incompletos.

Ejemplo:

```
{  
  "error": true,  
  "codigo": 400,  
  "mensaje": "El parametro mes debe estar entre 1 y 12"  
}
```

Error 404: datos no encontrados

Se usaría cuando no hay información para los filtros solicitados.

Ejemplo:

```
{  
  "error": true,  
  "codigo": 404,  
  "mensaje": "No se encontraron datos para la ruta y periodo solicitados"  
}
```

Error 500: error interno

Se usaría cuando ocurre un problema interno del servidor.

Ejemplo:

```
{
```

```
"error": true,  
  
"codigo": 500,  
  
"mensaje": "Ocurrio un error al procesar la solicitud"  
}
```

Relación entre endpoints y pantallas

Pantalla	Endpoints que utiliza
Inicio / Dashboard general	<code>/api/resumen</code> , <code>/api/evolucion-pasajeros</code> , <code>/api/alertas</code>
Rutas	<code>/api/rutas/top</code> , <code>/api/rutas/ocupacion</code> , <code>/api/rutas/baja-ocupacion</code> , <code>/api/rutas/detalle</code>
Predicción de demanda	<code>/api/prediccion/demanda</code> , <code>/api/prediccion/ocupacion</code> , <code>/api/filtros</code>
Alertas	<code>/api/alertas</code> , <code>/api/rutas/baja-ocupacion</code>
Mapa de aeropuertos	<code>/api/aeropuertos</code>
Filtros generales	<code>/api/filtros</code>

Relación entre endpoints y base de datos

Los endpoints principales deberían trabajar sobre una base limpia agregada por mes y ruta.

La estructura mínima esperada de esa base sería:

Columna	Uso
anio	Permite filtrar y agrupar por año.
mes	Permite analizar estacionalidad y evolución mensual.
ruta	Permite hacer rankings, filtros y predicciones por ruta.
clasificacion_vuelo	Permite separar cabotaje e internacional.
origen_localidad	Permite identificar el punto de salida.
origen_provincia	Permite analizar datos por provincia.
destino_localidad	Permite identificar el destino.
destino_provincia	Permite analizar destinos por provincia.

pasajeros	Variable central para medir demanda.
asientos	Variable central para medir oferta disponible.
vuelos	Permite analizar frecuencia u oferta de vuelos.
factor_ocupacion	Permite medir qué tan ocupada estuvo una ruta.

Justificación general del diseño de endpoints

La estructura propuesta permite dividir el sistema en partes claras.

Los endpoints de resumen y evolución alimentan el dashboard inicial. Esto permite que el usuario tenga una visión rápida del comportamiento general del transporte aéreo.

Los endpoints de rutas permiten analizar el proyecto desde un punto de vista más específico. En vez de mirar solamente datos globales, el sistema permite identificar qué rutas tienen más pasajeros, cuáles tienen mejor ocupación y cuáles presentan baja ocupación.

El endpoint de alertas permite transformar los datos en información útil para la toma de decisiones. Esto es importante porque el proyecto no busca ser solamente un tablero de gráficos, sino una herramienta inteligente que ayude a interpretar situaciones relevantes.

Los endpoints de predicción representan la parte de inteligencia artificial. Permiten estimar pasajeros y ocupación futura a partir de datos históricos, lo que se relaciona directamente con el objetivo principal del sistema.

El endpoint de filtros mejora la usabilidad, porque permite que el frontend construya menús y formularios con datos reales.

El endpoint de aeropuertos queda como opcional, ya que depende de si el equipo llega a implementar el mapa. Sin embargo, es una mejora interesante porque permite visualizar la conectividad aérea de forma geográfica.

Finalmente, el endpoint de métricas del modelo es opcional, pero ayuda a justificar la parte técnica de inteligencia artificial mostrando cómo fue evaluado el modelo.

Conclusión

Los endpoints propuestos cubren las necesidades principales de AeroPredict Argentina: mostrar información general, analizar rutas, calcular ocupación, detectar baja ocupación, generar alertas y realizar predicciones de demanda.

La API está pensada para trabajar con una base limpia y agregada por mes y ruta, lo que permite que el sistema sea más ordenado, más rápido y más fácil de mantener.

Con esta estructura, el frontend puede construir las pantallas principales del sistema y backend puede separar claramente las consultas históricas de las funciones predictivas. Además, el diseño deja margen para incorporar mejoras futuras, como mapas de aeropuertos, filtros avanzados y métricas del modelo de inteligencia artificial.